

```
1 package com.tp.jp.a.util;
2
3 import java.util.Scanner;
4
5 public class Input {
6     public static Double DoubleSeguro(Scanner sc,
7     String mensaje) {
8         while (true) {
9             System.out.print(mensaje);
10            String input = sc.nextLine();
11
12            try {
13                return Double.parseDouble(input);
14            } catch (NumberFormatException e) {
15                System.out.println("Debe ingresar un
16                número válido.");
17            }
18        }
19    }
20
21    public static Long LongSeguro(Scanner sc, String
22    mensaje) {
23        while (true) {
24            System.out.print(mensaje);
25            String input = sc.nextLine();
26
27            try {
28                return Long.parseLong(input);
29            } catch (NumberFormatException e) {
30                System.out.println("Debe ingresar un
31                número válido.");
32            }
33        }
34    }
35
36    public static int intSeguro(Scanner sc, String
37    mensaje) {
38        while (true) {
39            System.out.print(mensaje);
40            String input = sc.nextLine();
41
42            try {
43                return Integer.parseInt(input);
44            } catch (NumberFormatException e) {
45                System.out.println("Debe ingresar un
46                número válido.");
47            }
48        }
49    }
50}
```

```
37         try {
38             return Integer.parseInt(input);
39         } catch (NumberFormatException e) {
40             System.out.println("Debe ingresar un
    número válido.");
41         }
42     }
43 }
44 }
```

```
1 package com.tp.jp.util;  
2  
3 import jakarta.persistence.EntityManager;  
4 import jakarta.persistence.EntityManagerFactory;  
5 import jakarta.persistence.Persistence;  
6  
7 public class JPAUtil {  
8  
9     private static final String PERSISTENCE_UNIT = "  
miUnidad";  
10  
11     private static EntityManagerFactory emf;  
12  
13     static {  
14         try {  
15             emf = Persistence.  
createEntityManagerFactory(PERSISTENCE_UNIT);  
16         } catch (Exception e) {  
17             System.err.println("Error inicializando  
EntityManagerFactory: " + e.getMessage());  
18             throw new ExceptionInInitializerError(e);  
19         }  
20     }  
21  
22     public static EntityManager getEntityManager() {  
23         return emf.createEntityManager();  
24     }  
25  
26     //como usarlo -> EntityManager em = JPAUtil.  
getEntityManager();  
27  
28     public static EntityManagerFactory  
getEntityManagerFactory() {  
29         return emf;  
30     }  
31  
32     public static void close() {  
33         if (emf != null && emf.isOpen()) {  
34             emf.close();  
35         }  
36     }
```

37 }

38

```
1 package com.tp.jpa.util;
2
3 import com.tp.jpa.model.entities.Categoria;
4 import com.tp.jpa.model.entities.Producto;
5 import com.tp.jpa.repository.CategoriaRepository;
6 import com.tp.jpa.repository.ProductoRepository;
7
8 import java.util.List;
9
10 public class Reports {
11     public static void
12     listarProductoPorCategoriaHelper(ProductoRepository
13     productoRepo, long idReporte){
14
15         List<Producto> productos = productoRepo.
16         buscarPorCategoria(idReporte);
17
18         if (productos.stream().findAny().isEmpty()) {
19             System.out.println("No hay productos
20             activos.");
21         } else {
22             System.out.println("Productos de la
23             categoria: ");
24             productos.forEach(p ->
25                 System.out.println(
26                     "ID: " + p.getId() +
27                     " | Nombre: " + p
28                     .getNombre() +
29                     " | Precio: " + p
30                     .getPrecio() +
31                     " | Descripcion
32                     : " + p.getDescripcion() +
33                     " | Stock: " + p.
34                     getStock() +
35                     " | Categoría: "
36                     + p.getCategoria().getNombre()
37                     )
38             );
39         }
40     }
41 }
```

```
32     public static boolean mostrarProductosActivos(  
    ProductoRepository productoRepo) {  
33  
34         List <Producto> productos = productoRepo.  
listarActivos();  
35  
36         if (productos.stream().findAny().isEmpty()) {  
37             System.out.println("No hay productos  
activos.");  
38             return false;  
39         } else {  
40             productos.forEach(p ->  
41                 System.out.println(  
42                     "ID: " + p.getId() +  
43                     " | Nombre: " + p  
44                     .getNombre() +  
45                     " | Precio: " + p  
46                     .getPrecio() +  
47                     " | Descripcion  
: " + p.getDescripcion() +  
48                     " | Stock: " + p.  
getStock() +  
49                     " | Categoría: "  
+ p.getCategoria().getNombre()  
50                     )  
51                 );  
52             return true;  
53         }  
54     }  
55  
56     public static boolean mostrarCategoriasActivas(  
    CategoriaRepository categoriaRepo) {  
57  
58         List<Categoria> categorias = categoriaRepo.  
listarActivos();  
59  
60         if (categorias.stream().findAny().isEmpty  
()) {  
            System.out.println("No hay categorías  
activas.");  
            return false;  
        }  
    }
```

```
61         } else {
62             System.out.println("Categorias
disponibles :");
63             categorias.forEach(cat ->
64                 System.out.println(
65                     "ID: " + cat.getId() +
66                     " | Nombre: " +
cat.getNombre() +
67                     " | Descripcion
: " + cat.getDescripcion()
68                 )
69             );
70             return true;}
71     }
72
73 }
```

```
1 package com.tp.jpa.util;
2
3 import com.tp.jpa.model.entities.Categoria;
4 import com.tp.jpa.model.entities.Producto;
5 import com.tp.jpa.repository.CategoriaRepository;
6 import com.tp.jpa.repository.ProductoRepository;
7
8 import java.util.Optional;
9 import java.util.Scanner;
10
11 import static com.tp.jpa.util.Input.intSeguro;
12
13 public class Validator {
14
15     public static boolean validarNombreCategoria(
16         String nombre, CategoriaRepository categoriaRepo,
17         Scanner sc) {
18         //Validacion de nombre duplicado
19         boolean existe = categoriaRepo.listarActivos
20             ()
21             .stream()
22             .anyMatch(c -> c.getNombre().
23                 equalsIgnoreCase(nombre));
24
25         Optional<Categoria> categoriaInactiva =
26             categoriaRepo.listarInactivos()
27             .stream()
28             .filter(c -> c.getNombre().
29                 equalsIgnoreCase(nombre))
30             .findFirst();
31
32         //verificacion de categoria repetida
33         if (existe) {
34             System.out.println("Ya existe una
35                 categoría con ese nombre.");
36             return true; // no continua
37         }
38
39         //verificacion de categoria existente pero
40         con baja logica
41         if (categoriaInactiva.isPresent()) {
42             System.out.println("Ya existe una
```



```

33 categoría con ese nombre, pero se encuentra inactiva"
    +
34         "\n ¿Desea activarla nuevamente"
    );
35         System.out.println("1. Si");
36         System.out.println("2. No");
37         int opcionAlta = intSeguro(sc, "
    Seleccione una opción: ");
38
39         switch (opcionAlta) {
40             case 1 -> {
41                 Categoria cat = categoriaInactiva
42                     .get();
43                 categoriaRepo.AltaLogica(cat.
44                     getId());
45                 System.out.println("Categoría
    reactivada correctamente. ID: " + cat.getId());
46                 return true;
47             }
48             case 2 -> {
49                 System.out.println("Operación
    cancelada.");
50                 return true; //no continua
51             }
52         }
53     }
54     return false; //si pasa las evaluaciones
    continua con el alta
55 }
56
57
58 public static boolean validarNombreProducto(
    String nombre, ProductoRepository productoRepo,
    Scanner sc) {
59     boolean existe = productoRepo.listarActivos()
60         .stream()
61         .anyMatch(c -> c.getNombre().
    equalsIgnoreCase(nombre));
62

```

```

63         Optional<Producto> prodInactivo =
            productoRepo.listarInactivos()
64             .stream()
65             .filter(c -> c.getNombre().
                equalsIgnoreCase(nombre))
66             .findFirst();
67
68         //verificacion de producto repetido
69         if (existe) {
70             System.out.println("Ya existe un
producto con ese nombre.");
71             return true; // no continua
72         }
73
74         //verificacion de producto existente pero
            con baja logica
75         if (prodInactivo.isPresent()) {
76             System.out.println("Ya existe un
producto con ese nombre, pero se encuentra inactivo"
77             +
                "\n ¿Desea activarlo nuevamente"
78             );//
79             System.out.println("1. Si");
80             System.out.println("2. No");
81             int opcionAltaProd = intSeguro(sc, "
                Seleccione una opción: ");
82
83             switch (opcionAltaProd) {
84                 case 1 -> {
85                     Producto prod = prodInactivo.get
86                     ();
87                     productoRepo.AltaLogica(prod.
88                     getId());
89                     System.out.println("Producto
reactivado correctamente. ID: " + prod.getId());
90                     return true; // no continua
91                 }
92                 case 2 -> {
93                     System.out.println("Operación
cancelada.");
94                     return true; // no continua

```

```
92         }
93     }
94
95
96     }
97     return false; //si pasa las evaluaciones
    continua con el alta
98 }
99 }
100
```

```

1 package com.tp.jpa.util;
2
3 import com.tp.jpa.model.entities.Categoria;
4 import com.tp.jpa.model.entities.Producto;
5 import com.tp.jpa.repository.CategoriaRepository;
6 import com.tp.jpa.repository.ProductoRepository;
7
8 public class DataLoader {
9
10     public static void load(CategoriaRepository
        categoriaRepo, ProductoRepository productoRepo) {
11
12         //ESTA CLASE ES SOLO PARA PRUEBAS DEL CODIGO
        . CARGA CATEGORIAS Y PRODUCTOS SI LA BASE SE
        ENCUENTRA VACIA.
13         // Si ya existen categorías o productos
        cargados, no carga nada
14         if (!categoriaRepo.listarActivos().isEmpty
            () || !categoriaRepo.listarInactivos().isEmpty() || !
            productoRepo.listarActivos().isEmpty() || !
            productoRepo.listarInactivos().isEmpty()) {
15             return;
16         }
17
18         System.out.println("Cargando datos iniciales
        de hardware...");
19
20         // =====
21         // ===== 1. CATEGORÍAS =====
22         // =====
23         Categoria micro = Categoria.builder()
24             .nombre("Microprocesadores")
25             .descripcion("CPUs Intel y AMD")
26             .build();
27
28         Categoria mother = Categoria.builder()
29             .nombre("Motherboards")
30             .descripcion("Placas madre Intel y
        AMD")
31             .build();
32

```

```

33         Categoria memorias = Categoria.builder()
34             .nombre("Memorias RAM")
35             .descripcion("DDR4 y DDR5")
36             .build();
37
38         Categoria perifericos = Categoria.builder()
39             .nombre("Periféricos")
40             .descripcion("Teclados, mouse,
auriculares")
41             .build();
42
43         Categoria fuentes = Categoria.builder()
44             .nombre("Fuentes de alimentación")
45             .descripcion("PSU 500W a 1000W")
46             .build();
47
48         micro = categoriaRepo.guardar(micro);
49         mother = categoriaRepo.guardar(mother);
50         memorias = categoriaRepo.guardar(memorias);
51         perifericos = categoriaRepo.guardar(
perifericos);
52         fuentes = categoriaRepo.guardar(fuentes);
53
54         // =====
55         // 2. PRODUCTOS
56         // =====
57
58         // MICROPROCESADORES
59         productoRepo.guardar(Producto.builder().
nombre("Intel i3").descripcion("4 núcleos, 8 hilos").
precio(95000.0).stock(15).disponible(true).categoria(
micro).build());
60         productoRepo.guardar(Producto.builder().
nombre("Intel i5").descripcion("6 núcleos, 12 hilos"
).precio(150000.0).stock(10).disponible(true).
categoria(micro).build());
61         productoRepo.guardar(Producto.builder().
nombre("Intel i7").descripcion("12 núcleos, 20 hilos"
).precio(280000.0).stock(8).disponible(true).
categoria(micro).build());
62         productoRepo.guardar(Producto.builder().

```

```

62 nombre("Ryzen 5").descripcion("6 núcleos, 12 hilos"
    ).precio(130000.0).stock(12).disponible(true).
    categoria(micro).build());
63     productoRepo.guardar(Producto.builder().
        nombre("Ryzen 7").descripcion("8 núcleos, 16 hilos"
            ).precio(210000.0).stock(7).disponible(true).
            categoria(micro).build());
64
65     // MOTHERBOARDS
66     productoRepo.guardar(Producto.builder().
        nombre("ASUS Prime").descripcion("Intel 12va gen").
        precio(120000.0).stock(10).disponible(true).
        categoria(mother).build());
67     productoRepo.guardar(Producto.builder().
        nombre("Gigabyte B550M").descripcion("AM4 Ryzen").
        precio(110000.0).stock(14).disponible(true).
        categoria(mother).build());
68     productoRepo.guardar(Producto.builder().
        nombre("MSI B450").descripcion("AM4 Ryzen").precio(
        105000.0).stock(9).disponible(true).categoria(mother
        ).build());
69     productoRepo.guardar(Producto.builder().
        nombre("ASRock Z690").descripcion("Intel 12va gen").
        precio(220000.0).stock(5).disponible(true).categoria
        (mother).build());
70     productoRepo.guardar(Producto.builder().
        nombre("ASUS ROG").descripcion("AM4 Gaming").precio(
        180000.0).stock(6).disponible(true).categoria(mother
        ).build());
71
72     // MEMORIAS RAM
73     productoRepo.guardar(Producto.builder().
        nombre("Corsair 8GB").descripcion("3200MHz").precio(
        25000.0).stock(20).disponible(true).categoria(
        memorias).build());
74     productoRepo.guardar(Producto.builder().
        nombre("Corsair 16GB").descripcion("3200MHz").precio
        (45000.0).stock(18).disponible(true).categoria(
        memorias).build());
75     productoRepo.guardar(Producto.builder().
        nombre("HyperX 16GB").descripcion("3600MHz").precio(

```

```

75 50000.0).stock(15).disponible(true).categoria(
    memorias).build());
76     productoRepo.guardar(Producto.builder().
    nombre("Kingston 8GB").descripcion("5200MHz").precio
    (60000.0).stock(10).disponible(true).categoria(
    memorias).build());
77     productoRepo.guardar(Producto.builder().
    nombre("Kingston 16GB").descripcion("5600MHz").
    precio(90000.0).stock(8).disponible(true).categoria(
    memorias).build());
78
79     // PERIFÉRICOS
80     productoRepo.guardar(Producto.builder().
    nombre("Logitech G203").descripcion("Mouse gamer RGB
    ").precio(15000.0).stock(25).disponible(true).
    categoria(perifericos).build());
81     productoRepo.guardar(Producto.builder().
    nombre("Redragon K552").descripcion("Teclado
    mecánico").precio(30000.0).stock(20).disponible(true
    ).categoria(perifericos).build());
82     productoRepo.guardar(Producto.builder().
    nombre("HyperX Cloud").descripcion("Auriculares
    gamer").precio(35000.0).stock(12).disponible(true).
    categoria(perifericos).build());
83     productoRepo.guardar(Producto.builder().
    nombre("Logitech C270").descripcion("Webcam HD").
    precio(20000.0).stock(10).disponible(true).categoria
    (perifericos).build());
84     productoRepo.guardar(Producto.builder().
    nombre("Redragon M607").descripcion("Mouse gamer
    7200 DPI").precio(12000.0).stock(18).disponible(true
    ).categoria(perifericos).build());
85
86     // FUENTES
87     productoRepo.guardar(Producto.builder().
    nombre("Corsair CV550").descripcion("550W 80+ Bronze
    ").precio(35000.0).stock(10).disponible(true).
    categoria(fuentes).build());
88     productoRepo.guardar(Producto.builder().
    nombre("EVGA 600 BR").descripcion("600W 80+ Bronze"
    ).precio(40000.0).stock(8).disponible(true).

```

```
88 categoria(fuentes).build());
89     productoRepo.guardar(Producto.builder().
    nombre("Gigabyte P650B").descripcion("650W 80+
    Bronze").precio(45000.0).stock(7).disponible(true).
    categoria(fuentes).build());
90     productoRepo.guardar(Producto.builder().
    nombre("Thermaltake 700W").descripcion("700W 80+
    White").precio(50000.0).stock(6).disponible(true).
    categoria(fuentes).build());
91     productoRepo.guardar(Producto.builder().
    nombre("Corsair RM750").descripcion("750W 80+ Gold"
    ).precio(80000.0).stock(5).disponible(true).
    categoria(fuentes).build());
92
93     System.out.println("Datos iniciales de
    hardware cargados correctamente.");
94
95 }
96 }
97
```